

Software Requirements Specification

Student Club Event Ticketing and Budget Portal

Shishir Hegde | SRN PES1UG24CS438 | Section 5H

Version 1.0. This specification describes a small portal for club event proposals, budget approvals and attendee entry. It is based on Problem Statement 10 and the revised Lab 1 requirements. It also states what the submitted prototype does not cover.

1 Purpose and scope

The portal keeps the event proposal, approval record and ticket registrations together. A Club Lead prepares a proposal, university reviewers decide on the budget, students register for an approved event, and staff validate tickets at entry. The system does not collect ticket payments or transfer approved money.

2 Users and responsibilities

User	Responsibility
Club Lead	Create and edit own drafts, submit requests and track own events.
Faculty Coordinator	Perform the first budget review.
Finance Officer	Review requests cleared by the coordinator.
Dean	Give the final decision after finance approval.
Student	Register for an approved event and access their own ticket.
Check-in Staff	Validate QR ticket text for a selected approved event.

Campus Admin in the original brief is represented by the reviewer and check-in roles. One account has one role. Accounts and club allocations are provisioned separately; self-service account creation is outside the prototype.

3 Functional requirements

FR-001 (High): The system shall route submitted budget requests through Faculty Coordinator, Finance Officer and Dean in that order. Each decision shall record the authenticated reviewer, stage, decision and time. Rejection shall stop the workflow and require a reason.

FR-002 (High): The system shall let a Club Lead create and edit their own draft proposal with an event name, date, venue, capacity and itemised budget. Submission shall validate required fields and the club allocation, then reserve the requested amount.

FR-003 (High): The system shall issue a unique, signed QR ticket when a student registers for an approved, unexpired event with space available. The ticket shall identify the event and attendee. Repeat registration shall return the existing ticket.

FR-004 (High): The system shall allow check-in staff to validate ticket text read from a QR code, accept an unused valid ticket once and mark it used. It shall reject altered, malformed, expired, wrong-event and previously used tickets.

FR-005 (Medium): The system shall show a Club Lead the current approval status, registration count and remaining allocation for their own events. The dashboard shall refresh status and counts within 5 seconds of a committed change during normal connected use.

The complete acceptance criteria and rationale are in requirements-and-use-case.pdf. The RTM maps each requirement to code and test cases.

4 Approval states

Draft -> Faculty pending -> Finance pending -> Dean pending -> Approved. At any pending stage, the assigned reviewer may choose Rejected. Approved and Rejected are final states for this prototype. A rejected request is not silently reopened; a lead can prepare a new proposal.

5 Business rules

BR01. Only the owning Club Lead may edit or submit a draft. Submitted event details are locked.

BR02. Event name, date, venue, positive capacity and at least one positive budget item are required. Dates before the server's current date are refused.

BR03. Amounts are entered in rupees with at most two decimal places and stored as integer paise. The total equals the sum of the items.

BR04. Pending and approved budgets count against one configured allocation period. The check and reservation occur together. Rejection releases the reservation.

BR05. Reviewers act only at their assigned stage. A sign-off records account, stage, decision and UTC time. Rejection needs a reason.

BR06. A student has at most one ticket per event. Registration is limited by capacity and final approval.

BR07. Tickets expire after the event date in the server's configured local time zone. There is no separate time-of-day entry window in this version.

BR08. A valid ticket can be consumed once, even if two staff requests arrive together. Only check-in accounts may do this.

BR09. Dashboard data is restricted by role and ownership. The page polls every three seconds while open.

BR10. The minimum guarantee on any failed operation is no partial budget reservation, approval or ticket consumption.

6 Non-functional requirements

NFR-001 targets complete check-in validation below 100 ms with staff authorisation, signature verification and replay rejection. The initial peak-load assumption is 20 simultaneous check-in clients.

NFR-002 targets 500 concurrent users, page loads below two seconds and usable mobile forms. These are targets for acceptance; the test report records the limits of the local measurements.

7 Data design

Entity	Main fields and constraints
Club	ID, name, allocation in paise. A lead is assigned to a club.
User	ID, unique username, password hash, role and optional club ID.
Event	ID, club, owner, name, date, venue, capacity, budget and state.
Budget item	ID, event ID, description and positive amount.
Approval	ID, event ID, reviewer, stage, decision, reason and UTC time. One recorded decision per event/stage.
Ticket	Random ID, event ID, attendee ID and optional used time. Unique event/attendee pair.

SQLite foreign keys enforce links between records. Parameterised SQL is used for all user values. Write operations use explicit transactions. Ticket use is updated only where used_at is still empty.

8 Security and privacy

The application uses password hashes, signed session cookies, CSRF tokens on state-changing forms, role checks and ticket HMAC verification. Template escaping prevents event text being interpreted as HTML. The ticket secret and database are local instance files and are excluded from Git.

The demonstration accounts and password are only for local use. Internet deployment would require replacing them, enabling HTTPS and secure cookies, adding login rate limits and defining backups and retention. QR contents use internal IDs rather than names or email addresses.

9 External interfaces

Interface	Input and output
Browser pages	HTML forms for login, proposals, reviews, registration and check-in.
GET /api/status	Authenticated request; JSON containing visible event states, counts and remaining allocation.
POST /events/new or /events/{id}/edit	Lead form with event fields and budget lines; saves a draft or returns a validation message.
POST /events/{id}/submit	Lead request; reserves allocation and changes state.
POST /events/{id}/review	Reviewer decision and reason; stores sign-off or rejects an invalid stage.
POST /events/{id}/register	Student request; redirects to the issued or existing ticket.
GET /tickets/{id} and .svg	Own-ticket page and QR image; another student receives no ticket data.
POST /checkin	Staff account, selected event and scanned ticket text; displays acceptance or rejection.

Every POST form includes a CSRF token. Missing or invalid tokens return HTTP 400. Unauthenticated browser requests go to login; denied pages return 403 or 404. Business-rule failures show a plain message without committing an invalid change.

10 Assumptions and exclusions

The university will supply the real allocation policy, account provisioning method and expected load. The prototype uses one allocation period; annual rollover and spending reconciliation are future work. An authenticated decision is the digital sign-off; certificate signing is not included.

A keyboard-type QR scanner can supply ticket text to the check-in field. The current evidence uses manually pasted ticket text. There is no camera scanner, email delivery, payment gateway, attendance export or deployed university integration.

11 Verification and acceptance

Automated tests cover invalid proposals, approval order, rejection, budget reservation, duplicate registration, capacity, ownership, ticket tampering, replay and concurrent updates. A browser walkthrough covers the normal flow. A separate script measures local HTTP load and check-in time.

Acceptance requires the functional cases to pass and the non-functional targets to be verified on the intended environment. The present prototype is a local demonstration, not a claim of production readiness. See the test report for actual results and remaining checks.

12 References

Problem Statement 10: 10_SE_Lab1_SE_Problem_Statements.pdf.

Git Hub Project Submission Details.docx, individual-project section.

Lab_3_Architecture_Student_handout.pdf, component diagram and justification requirements.

Revised requirements, RTM and architecture files in folders 1 and 2 of this repository.